

Chat Protocol Specification

1. Overview

This document specifies a Key-Length-Value (KLV) based binary protocol for a multi-client chat application. The protocol supports user authentication, message broadcasting, message history retrieval, and user disconnection.

1.1 KLV Format

KLV (Key-Length-Value) is a binary encoding format where each message consists of three parts:

- **Key:** 4-byte fixed-width ASCII string (see section 1.4)
- **Length:** 4-byte unsigned integer (uint32, big-endian) indicating the number of bytes in the value field
- **Value:** Variable-length message data (may contain nested KLV structures)

1.2 Documentation Notation

Important: For readability, this specification uses a colon-separated text format to represent binary messages. **In the actual wire format, keys are 4-byte ASCII strings and lengths are 4-byte integers** - no delimiters exist.

Documentation format: JOIN:5:alice

Wire format: [JOIN][0x00000005][alice]

Where 'JOIN' is the 4-byte key, 0x00000005 is the 4-byte length, and 'alice' is the 5-byte value.

1.3 Byte Order

All multi-byte integers (length fields) use **big-endian** byte order (network byte order).

1.4 Message Type Identifiers

All keys are exactly 4 bytes, using ASCII characters. Keys shorter than 4 characters are padded with null bytes (0x00).

Key	Bytes	Hex Representation	Description
JOIN	4	0x4A4F494E	User authentication
MSG\0	4	0x4D534700	Chat message (MSG + null padding)
READ	4	0x52454144	Retrieve message history

EXIT	4	0x45584954	User disconnect
RESP	4	0x52455350	Server response
FROM	4	0x46524F4D	Nested: message sender
BODY	4	0x424F4459	Nested: message content
CODE	4	0x434F4445	Nested: status code
TYPE	4	0x54595045	Nested: error description
MSGs	4	0x4D534753	Nested: message array

1.5 Status Codes

Code	Meaning	Description
200	Success	Request completed successfully
400	Invalid Message	Malformed request, empty message, or client error
403	User Taken	Username is already in use
503	Connection Failed	Server cannot process request (rarely used)

Note: Status codes are ASCII digit strings (e.g., '200' = 3 bytes, '403' = 3 bytes).

2. Message Types

2.1 JOIN - User Authentication

Purpose: Authenticate a user and add them to the chat room.

Client → Server Request:

JOIN:length:username

Example:

JOIN:5:alice

Wire format: [JOIN][0x00000005][alice]

Server → Client Response:

On success:

RESP:11:CODE:3:200

Wire format: [RESP][0x0000000B][CODE][0x00000003][200]

If username already taken:

RESP:41:CODE:3:403:TYPE:22:Username already taken

Server → All Clients Broadcast:

JOIN:length:username

Example:

JOIN:5:alice

Note: The server broadcasts the JOIN message to all connected clients (including the joining user) to notify them of the new participant.

2.2 MSG - Send Chat Message

Purpose: Send a message to all users in the chat room.

Client → Server Request:

MSG:length:FROM:length:username:BODY:length:text

Example:

MSG:32:FROM:5:alice:BODY:11:Hello World

Breaking down the structure:

• **MSG value (32 bytes):**

- FROM key (4) + FROM length (4) + 'alice' (5) = 13 bytes

- BODY key (4) + BODY length (4) + 'Hello World' (11) = 19 bytes

Server → Client Response:

On success:

RESP:11:CODE:3:200

On error:

RESP:37:CODE:3:400:TYPE:18:Empty message body

Server → All Clients Broadcast:

MSG:length:FROM:length:username:BODY:length:text

Note: The server broadcasts the message to all connected clients in the same format as received.

2.3 READ - Retrieve Message History

Purpose: Retrieve the last 20 messages (or all messages if fewer than 20 exist).

Client → Server Request:

READ:0:

Wire format: [READ] [0x00000000]

Note: The READ request has a zero-length value as it requires no parameters.

Server → Client Response:

Example with 3 messages:

RESP:127:CODE:3:200:MSGs:108:MSG:28:FROM:3:bob:BODY:9:Hey Alice:MSG:28:FROM:5:alice:BODY:7:Hey Bob:MSG:28:FROM:3:bob:BODY:9:How are u

Breaking down the structure:

- **RESP value (127 bytes)** contains CODE + MSGS
 - CODE key (4) + CODE length (4) + '200' (3) = 11 bytes
 - MSGS key (4) + MSGS length (4) + MSGS value (108) = 116 bytes
- **MSGs value (108 bytes)** contains 3 MSG structures:
 - MSG#1: MSG key (4) + MSG length (4) + MSG value (28) = 36 bytes
MSG value: FROM(4) + len(4) + 'bob'(3) + BODY(4) + len(4) + 'Hey Alice'(9) = 28
 - MSG#2: MSG key (4) + MSG length (4) + MSG value (28) = 36 bytes
MSG value: FROM(4) + len(4) + 'alice'(5) + BODY(4) + len(4) + 'Hey Bob'(7) = 28
 - MSG#3: MSG key (4) + MSG length (4) + MSG value (28) = 36 bytes
MSG value: FROM(4) + len(4) + 'bob'(3) + BODY(4) + len(4) + 'How are u'(9) = 28

Without messages (empty history):

RESP:11:CODE:3:200

Note: When messages exist, the response includes a MSGS field containing up to 20 MSG structures. Each MSG follows the standard format. Multiple MSG structures are concatenated within the MSGS value.

2.4 EXIT - User Disconnect

Purpose: Notify the server that a user is disconnecting.

Client → Server Request:

EXIT:length:username

Example:

EXIT:5:alice

Wire format: [EXIT] [0x00000005] [alice]

Server → Client Response:

On success:

RESP:11:CODE:3:200

Server → All Clients Broadcast:

EXIT:length:username

Example:

EXIT:5:alice

Note: The server broadcasts the EXIT message to all remaining clients to notify them that a user has disconnected.

3. Complete Message Flow Examples

3.1 User Joining

1. Alice connects and sends JOIN:
`JOIN:5:alice`
2. Server validates and responds:
`RESP:11:CODE:3:200`
3. Server broadcasts to all clients (including Alice):
`JOIN:5:alice`

3.2 Sending a Message

1. Bob sends a message:
`MSG:26:FROM:3:bob:BODY:9:Hey Alice`
2. Server responds to Bob:
`RESP:11:CODE:3:200`
3. Server broadcasts to all clients:
`MSG:26:FROM:3:bob:BODY:9:Hey Alice`

3.3 Reading Message History

1. Charlie joins and wants to see previous messages:
`READ:0:`
2. Server responds with last messages:
`RESP:length:CODE:3:200:MSGs:totalLength:MSG:26:FROM:3:bob:BODY:9:Hey Alice:MSG:...`

3.4 User Leaving

1. Alice disconnects:
`EXIT:5:alice`
2. Server responds:
`RESP:11:CODE:3:200`
3. Server broadcasts to remaining clients:
`EXIT:5:alice`

4. Quick Reference

Message Type	Direction	Format
JOIN	Client → Server	JOIN:length:username
	Server → Client	RESP:11:CODE:3:200 or RESP:41:CODE:3:403:TYPE:22:...
	Server → All	JOIN:length:username
MSG	Client → Server	MSG:length:FROM:length:username:BODY:length:text
	Server → Client	RESP:11:CODE:3:200 or RESP:37:CODE:3:400:TYPE:18:...
	Server → All	MSG:length:FROM:length:username:BODY:length:text
READ	Client → Server	READ:0:
	Server → Client	RESP:11+:CODE:3:200:MSG:length:MSG:...:MSG:...
EXIT	Client → Server	EXIT:length:username
	Server → Client	RESP:11:CODE:3:200
	Server → All	EXIT:length:username

5. Implementation Notes

5.1 Key Encoding

All keys are exactly 4 bytes. Keys shorter than 4 characters must be padded with null bytes (0x00). For example, 'MSG' becomes [0x4D, 0x53, 0x47, 0x00].

5.2 Length Calculation

When calculating the length field for nested KLV structures, count only the bytes in the value field (after the outer key and length). In the wire format, this includes nested keys (4 bytes each), nested lengths (4 bytes each), and the actual data.

Example for MSG:

MSG:32:FROM:5:alice:BODY:11:Hello World

Wire format breakdown: [MSG\0] + [0x00000020] + [FROM] + [0x00000005] + [alice] + [BODY] + [0x0000000B] + [Hello World]

The length 32 (0x20) = FROM key (4) + FROM length (4) + 'alice' (5) + BODY key (4) + BODY length (4) + 'Hello World' (11).

5.3 Parsing Strategy

To parse a KLV message: (1) Read 4 bytes for the key, (2) Read 4 bytes for the length, (3) Read exactly 'length' bytes for the value. If the value contains nested KLV structures, recursively parse them

using the same strategy.

5.4 Error Handling

Servers should always send a RESP to every client request before broadcasting. Clients should wait for and parse responses to detect errors. Handle malformed messages gracefully.

5.5 Threading and Concurrency

Servers must handle multiple simultaneous client connections. Use multithreading with one thread per client connection, plus appropriate synchronization for shared resources (e.g., client list, message history).